



MATH307
Fast Fourier Transform (FFT) Implementation and Its Applications.

Contents

1	Abstract	2
2	Introduction	2
3	Problem Definition	2
4	Methodology	3
4.1	Algorithm Implementation	3
4.2	Computational Analysis	3
4.3	Simulation Tools	3
4.4	Applications	4
4.4.1	Signal Filtering and Noise Reduction	4
4.4.2	Frequency Analysis and Feature Detection	4
4.4.3	Image Processing (AI-Focused Application)	4
5	Results	4
6	References	4

1 Abstract

The Fast Fourier Transform (FFT) is a fundamental numerical algorithm that efficiently converts signals from the time domain to the frequency domain, significantly improving upon the classical Discrete Fourier Transform (DFT), which has a computational complexity of $O(N^2)$. By exploiting symmetry and recursive decomposition, the FFT reduces this complexity to $O(N \log N)$, making it suitable for large-scale computational problems.

This project focuses on implementing and analyzing the FFT as a numerical method, comparing it with the analytical DFT in terms of computational efficiency and numerical accuracy. In addition, the study explores practical applications of FFT in signal processing, frequency analysis, and image processing. Special emphasis is placed on image processing due to its relevance in artificial intelligence, particularly in tasks such as filtering, compression, and feature extraction.

Using Python-based simulations, the study evaluates algorithm performance under varying conditions such as signal size and noise levels. The results demonstrate the FFT's critical role as a foundational tool in modern numerical analysis and AI-related applications.

2 Introduction

Frequency-domain analysis is a fundamental tool in engineering, physics, and computer science. The Discrete Fourier Transform (DFT) provides a mathematical framework for decomposing a discrete signal into its frequency components. For a signal of length N , the DFT is defined as:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N}, \quad k = 0, 1, \dots, N-1 \quad (1)$$

Despite its theoretical importance, the DFT suffers from a computational complexity of $O(N^2)$, which becomes impractical for large datasets. The Fast Fourier Transform (FFT), introduced by Cooley and Tukey, addresses this limitation by recursively decomposing the DFT into smaller subproblems, reducing the complexity to $O(N \log N)$.

The FFT has become a cornerstone in modern computational systems, enabling real-time signal processing, efficient data compression, and large-scale numerical simulations. Its applications extend to artificial intelligence, where frequency-domain representations are widely used in preprocessing, feature extraction, and image enhancement.

This project investigates the FFT as a numerical method, bridging theoretical formulation with practical implementation and real-world applications.

3 Problem Definition

The objective of this project is to efficiently compute the frequency representation of discrete signals and images while minimizing computational cost and maintaining numerical accuracy.

Given a discrete signal:

$$x = \{x_0, x_1, \dots, x_{N-1}\}$$

the DFT is defined as:

$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N} \quad (2)$$

The computational challenge arises from the quadratic complexity of this formulation. The FFT resolves this by dividing the problem into smaller subproblems using even-odd decomposition, significantly improving performance.

For two-dimensional data (images), the transformation extends to:

$$X(u, v) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} x(m, n) e^{-i2\pi(\frac{um}{M} + \frac{vn}{N})} \quad (3)$$

Key variables include:

- x_n : input signal samples
- X_k : frequency-domain representation
- N : number of samples
- M, N : image dimensions
- i : imaginary unit

The problem is therefore to achieve efficient computation while enabling applications such as filtering, noise reduction, and feature extraction.

4 Methodology

The project follows a numerical and experimental approach consisting of implementation, analysis, and application.

4.1 Algorithm Implementation

- Implement the DFT directly using its mathematical definition
- Implement the FFT using the Cooley–Tukey recursive algorithm
- Extend the FFT to two-dimensional data for image processing

4.2 Computational Analysis

- Measure execution time for DFT and FFT for varying input sizes
- Analyze time complexity and verify $O(N \log N)$ behavior
- Compare numerical accuracy between DFT and FFT outputs

4.3 Simulation Tools

- Python as the main programming environment
- NumPy for numerical operations and FFT validation
- Matplotlib for visualization
- OpenCV/PIL for image processing tasks

4.4 Applications

4.4.1 Signal Filtering and Noise Reduction

Signals are transformed into the frequency domain, where noise components are removed using filters. The cleaned signal is reconstructed using the inverse FFT.

4.4.2 Frequency Analysis and Feature Detection

FFT is used to identify dominant frequency components, enabling analysis of periodic behavior in engineering and scientific applications.

4.4.3 Image Processing (AI-Focused Application)

The FFT is extended to two-dimensional data, allowing images to be analyzed in the frequency domain. Applications include:

- Noise removal using low-pass filters
- Edge enhancement using high-pass filters
- Image compression by removing insignificant frequencies

These techniques are fundamental in artificial intelligence pipelines, particularly in feature extraction and preprocessing for deep learning models.

5 Results

Preliminary results indicate a significant reduction in execution time when using FFT compared to the direct DFT, especially for large input sizes. Initial image processing experiments demonstrate effective noise reduction while preserving key structural features.

Further results will include:

- Execution time comparisons
- Error analysis between DFT and FFT
- Visual results of image filtering and enhancement

6 References

References

- [1] J. W. Cooley and J. W. Tukey, "An Algorithm for the Machine Calculation of Complex Fourier Series," *Mathematics of Computation*, 1965.
- [2] A. V. Oppenheim and R. W. Schaffer, *Discrete-Time Signal Processing*, Pearson, 2010.
- [3] S. W. Smith, *The Scientist and Engineer's Guide to Digital Signal Processing*, California Technical Publishing, 1997.
- [4] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, Pearson, 2018.
- [5] NumPy Documentation, FFT Module.
- [6] OpenCV Documentation, Image Processing.